THE

Capability Architecture

Toward a Computational Theory of Organizations

VOLUME I / FOUNDATIONS

SEAN BURMAN

Working Draft v0.3 | July 2026

WORKING MANUSCRIPT

This document is an exploratory technical reference, not a finalized standard. Definitions, principles, and hypotheses are versioned so they can be tested, corrected, and replaced without obscuring the intellectual history of the work.

The manuscript develops the conceptual outline contained in the project discovery notes that initiated The Capability Architecture. The platform discussed throughout is treated as a possible implementation of the theory, not as the theory itself. [1]

Citation style: numbered references. The bibliography includes only sources used in the manuscript. Original propositions are explicitly labeled as principles, hypotheses, or research questions.

Copyright © 2026 Sean Burman. Working draft for review and research discussion.

PREFACE

Why a New Architecture?

Business software has reached a peculiar maturity. It is more capable, connected, and available than at any point in its history, yet the basic relationship between software and the organization remains largely unchanged. A workflow is specified, an application is built, and people are expected to fit their work into the application. When reality changes, the software changes only after someone notices the divergence, writes a requirement, funds a project, and waits for a new release.

Organizations do not behave like finished specifications. They are living arrangements of people, resources, obligations, habits, exceptions, and negotiated authority. Their practical intelligence is distributed across conversations, routines, tools, and local knowledge. Much of what makes an organization effective is never fully represented in its databases. It survives in the heads of experienced people and in the workarounds that formal systems fail to see. Simon described decision makers as operating under bounded rationality; Hayek emphasized that useful knowledge is dispersed among individuals and circumstances; cybernetics studied how systems regulate themselves through communication and feedback. [2][5][6][13]

The central question of this book is therefore not how to build another category of enterprise application. It is: how should an intelligent organization be represented computationally, so that software can observe the organization, understand its changing structure, and develop new capabilities in response? [1]

The word capability is used deliberately. Automation performs a known procedure. A capability is a persistent ability to achieve a class of outcomes. It may combine knowledge, authority, data, software, human judgment, and tools. A capability can be improved, composed with other capabilities, transferred between contexts, measured, governed, and retired. The architecture developed here treats capability - rather than the application, dashboard, prompt, or model - as the fundamental unit of adaptive business software.

This volume begins from first principles. Technology choices appear near the end because Go, React, containers, databases, and language models are implementation materials. They matter, but they should not decide what an organization is, what an observation means, or what authority an agent possesses. The theory must remain coherent even as those tools change.

PRINCIPLE P-001

Theory precedes implementation

The implementation may validate, refine, or falsify the theory, but implementation convenience must not silently define the theory.

PRINCIPLE P-002

Capability is the product

Data, information, dashboards, and recommendations are intermediate forms. The durable product is an improved organizational ability to achieve outcomes.

This is not a manifesto and it is not marketing copy. Every major claim is intended to be traceable to established literature, empirical observation, logical derivation, or an explicitly identified hypothesis. The objective is to develop a vocabulary and architecture that can be criticized, tested, and implemented.

HOW TO READ THIS WORK

A Versioned Body of Knowledge

The manuscript distinguishes different kinds of claims. This prevents a compelling idea from being presented with the authority of an established result and allows the work to evolve without rewriting its history.

DEFINITION

Definition

A term assigned a precise meaning within this work. Definitions are normative for interpreting the manuscript, but may be revised between versions.

PRINCIPLE

Principle

A foundational design commitment adopted by the architecture. A principle is stronger than a preference but may remain open to challenge.

HYPOTHESIS

Hypothesis

A proposition that appears useful or plausible but requires evidence, experimentation, or formal analysis.

OBSERVATION

Observation

An empirical statement about organizations, work, or software derived from direct experience, case material, or external research.

IMPLEMENTATION PATTERN

Implementation Pattern

A practical structure that realizes one or more principles. Patterns are replaceable and technology-sensitive.

RESEARCH QUESTION

Research Question

An unresolved problem whose answer may alter the theory or architecture.

The keywords MUST, MUST NOT, SHOULD, SHOULD NOT, and MAY are used in the normative sense established by RFC 2119 when they appear in uppercase. [29]

CONTENTS

Volume I - Foundations

Preface - Why a New Architecture?....3
How to Read This Work....5
Index of Foundational Claims....7
Chapter 1 - Purpose....9
Chapter 2 - Philosophy....12
Chapter 3 - Definitions....15
Chapter 4 - Mathematical Models....20
Chapter 5 - Computational Models....23
Chapter 6 - Organizational Models....26
Chapter 7 - Capability Models....29
Chapter 8 - Agent Models....33
Chapter 9 - Architecture....37
Chapter 10 - Implementation....41
Conclusion - From Software Products to Capability Systems....45
Appendix A - Glossary....46
Appendix B - Hypothesis Register....48
Works Cited....50
Revision Notes....52

INDEX OF FOUNDATIONAL CLAIMS

Definitions, Principles, and Hypotheses

Principles Introduced in Volume I

ID	Principle
P-001	Theory precedes implementation
P-002	Capability is the product
P-003	Software should evolve from observation rather than specification alone
P-004	Organizational representations must preserve provenance and time
P-005	Authority must be explicit and machine-enforceable
P-006	Agents receive bounded autonomy, not general permission
P-007	Every generated capability requires measurable outcomes and a retirement path
P-008	The organizational model is plural: event, graph, document, and analytical views must coexist
P-009	Human purpose remains external to optimization
P-010	The system must make uncertainty visible rather than conceal it

Hypotheses Introduced in Volume I

ID	Hypothesis	Status
H-001	Organizations can be represented usefully as dynamic, temporal knowledge graphs.	Under investigation
H-002	Capability is a stronger unit of reusable organizational intelligence than application feature or workflow.	Tentatively supported
H-003	Agents are runtime realizations of capability definitions.	Speculative
H-004	Structured observation can reveal software requirements that users cannot articulate directly.	Under investigation

H-005	Most persistent operational friction appears first as repeated exception handling.	Under investigation
H-006	The economic value of an agent is primarily encoded organizational judgment, not model access.	Speculative
H-007	A capability registry can support composition, licensing, and cross-industry adaptation.	Speculative
H-008	Outcome-linked event histories can reduce the cost of organizational learning.	Under investigation

CHAPTER 01

Purpose

Why this work exists and what problem it is attempting to solve.

Purpose

The purpose of The Capability Architecture is to define a computational framework for organizations that can observe their own operation, construct and revise models of how work occurs, and develop governed software capabilities in response. The work is broader than a product architecture because the target is not a particular application. The target is a general relationship between organizations, software, and intelligence. [1]

The immediate engineering problem is familiar: business requirements are incomplete, workflows contain exceptions, and users often cannot describe the system they need until they interact with a partial implementation. Traditional requirements gathering treats this as a failure of elicitation. The Capability Architecture treats it as evidence that the organization is not fully knowable in advance. The architecture therefore begins with observation and revision, not with the assumption of a final specification.

PRINCIPLE P-003

Software should evolve from observation rather than specification alone

Specifications remain useful, but the operational system must be able to learn from actual work, detected friction, exceptions, and outcomes.

The Motivating Problem

A static application contains an encoded theory of the organization. Its forms imply which entities matter. Its permissions imply who is trusted. Its workflow states imply which transitions are legitimate. Its reports imply what success means. These assumptions are often invisible until reality violates them. A customer changes a process, a regulation adds a constraint, a skilled employee leaves, or a project encounters an exception. The software does not become aware of the change; it merely becomes wrong.

Organizations have always adapted through people. Experienced workers notice discrepancies, invent workarounds, teach newcomers, and negotiate new routines. That adaptation is expensive because it is largely informal. It can disappear when staff leave and is difficult to transfer between teams. Learning-organization theory emphasizes the importance of turning experience into shared understanding, while evolutionary theories of the firm describe routines as carriers of organizational knowledge. [8][15]

The architectural opportunity is to make more of this adaptive process computationally legible without pretending that human judgment can be reduced to a complete rule set. The system should observe changes, preserve their context, identify recurrence, and help convert successful responses into persistent capability.

Scope

Volume I focuses on the foundations needed to reason about such a system. It addresses vocabulary, formal representations, organizational structure, capability, agents, architecture, and a reference implementation. It does not claim to solve general intelligence, fully automate management, or create a complete mathematical theory of organizations. It proposes a disciplined starting point.

- In scope: organizational observations, events, state, workflows, goals, constraints, authority, memory, capability definitions, agent runtimes, governance, evaluation, and implementation structure.
- Out of scope: replacing human purpose, inferring moral goals from operational data, unrestricted autonomous management, and treating model output as organizational truth.
- Long-term target: systems that can discover, propose, implement, evaluate, and revise new software capabilities under explicit human authority.

The Platform as First Implementation

Separating the theory from the platform protects both. The theory can be challenged without destabilizing a product roadmap, and the platform can adopt better technology without rewriting the meaning of its core concepts. Domain-driven design similarly separates a model of the domain from incidental infrastructure, while RFC practice separates durable protocol semantics from particular implementations. [20][29]

The first platform need not generate entire applications. Its initial role is to create a faithful observation layer: a registry of work objects, a canonical event history, explicit outcomes, structured exceptions, and a model of relationships. Capability generation is a later consequence of that foundation.

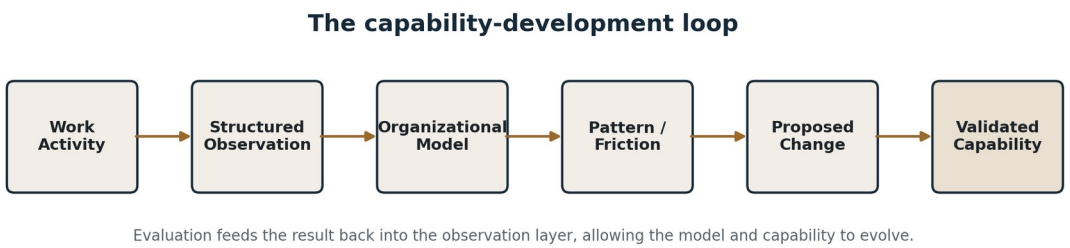


Figure 1.1 - The system develops capability by closing a loop between work, observation, modeling, change, and evaluation.

Success Criteria

The architecture succeeds only if it improves organizational ability rather than merely increasing data collection. A successful implementation should reduce the effort required to understand current work,

preserve knowledge that would otherwise disappear, identify recurring friction, support safer automation, and measure whether the resulting capability produces better outcomes.

HYPOTHESIS H-004

Observation can reveal unarticulated requirements

Repeated behavior, exceptions, rework, and local adaptations may expose requirements that interviews and specifications fail to capture.

Chapter Summary

The Capability Architecture exists to address a mismatch: organizations continuously adapt while most software changes only through explicit development projects. The proposed response is an observation-driven architecture whose durable output is new capability.

Open Research Questions

- Which classes of work can be observed without distorting the work itself?
- How much structure must exist before useful capability discovery becomes possible?
- When does an observed workaround indicate a missing capability rather than local preference?
- How can the system preserve the benefits of informal adaptation without normalizing unsafe behavior?

KEY SOURCES

This chapter draws primarily on [1], [8], [15], [20], [29].

CHAPTER 02

Philosophy

First principles governing purpose, intelligence, augmentation, and human authority.

Purpose Before Optimization

Organizations exist to pursue purposes that originate outside their software. A construction company builds and repairs physical infrastructure. A clinic improves health. A farm produces food. A research institution develops knowledge. Software is instrumental: it should help people coordinate resources and judgment toward those purposes. The architecture therefore rejects the idea that engagement, data accumulation, or model activity is intrinsically valuable.

> **PRINCIPLE P-009**
>
> **Human purpose remains external to optimization**
>
> The system may optimize methods and resources, but it must not silently redefine the organization's purpose from the behavioral data it observes.

This distinction matters because any adaptive system will discover proxies. Faster task closure may correlate with productivity, but it can also conceal poor quality. Lower labor hours may reflect efficiency or understaffing. More user interaction may indicate usefulness or confusion. The architecture must preserve a separation between observed metrics, inferred objectives, and authorized goals.

Data Is Material, Not Product

The discovery notes propose a progression from data to information, insight, recommendation, and tool creation. The deeper claim is that each intermediate product becomes valuable only when it improves the organization's ability to act. [1]

```
data -> information -> insight -> decision -> capability
```

Figure 2.1 - A conceptual progression, not a claim that each stage is fully automatable.

A dashboard can make a pattern visible, but the organization still depends on someone to interpret it repeatedly. A recommendation can tell a manager what to do, but the same reasoning must be repeated the next time. A capability incorporates the successful reasoning into a durable arrangement of knowledge, authority, process, and tools. It lowers the marginal effort required to achieve similar outcomes in the future.

Augmentation Rather Than Substitution

Engelbart framed computing as a means to augment human intellect by improving the integrated system of people, concepts, methods, and tools. Licklider imagined a symbiotic relationship in which humans and computers contribute complementary strengths. These traditions are more useful here than a narrow narrative of replacing workers. [9][10]

The architecture should remove avoidable cognitive and coordinative load while preserving meaningful judgment. It may assemble context, detect contradictions, simulate alternatives, and execute authorized procedures. It should also expose uncertainty, ask for clarification, and stop when authority or evidence is insufficient.

PRINCIPLE P-010

Uncertainty must remain visible

An intelligent system must represent confidence, missing context, disagreement, and model limits. Hidden uncertainty converts assistance into false authority.

Feedback and Learning

Cybernetics studies systems that regulate behavior through communication and feedback. The Capability Architecture adopts feedback but broadens the object being regulated. The goal is not merely to maintain a fixed target. Organizations may revise goals, constraints, roles, and capabilities as they learn. [2][4]

A capability therefore belongs inside a feedback loop. It is proposed because of observed friction, deployed under a defined mandate, evaluated against outcomes, and revised or retired. Without this loop, generated software becomes another layer of static infrastructure.

The Ethics of Observation

An observation layer creates power. It can expose performance, infer routines, and reconstruct decisions. The same architecture that helps an organization learn can become a mechanism for intrusive surveillance or automatic discipline. Ethical design is therefore structural, not cosmetic.

- Observations SHOULD be collected for declared operational purposes.
- Individuals SHOULD be able to distinguish recorded fact from system inference.
- Access to sensitive organizational memory MUST follow explicit authority.
- Automated evaluation MUST NOT collapse complex human performance into a single proxy metric.
- Generated capabilities MUST identify accountable owners and appeal paths.

Counterexample: The Omniscient Optimization Engine

Imagine a platform that observes every communication, infers employee goals, continuously ranks performance, and autonomously rewrites procedures to maximize a corporate metric. This may appear to be the logical conclusion of adaptive software. It is instead a counterexample. The system lacks legitimate authority to define purpose, confuses measurable behavior with value, and removes the possibility of contesting its model. It optimizes the representation rather than serving the organization.

Chapter Summary

The philosophy of the architecture is human-centered but not anti-automation. It treats computing as an instrument of organizational augmentation, data as material, capability as the durable output, and human authority as the source of legitimate goals.

Open Research Questions

- How should consent and notice operate when observations are generated indirectly from work systems?
- Which forms of organizational judgment should never be delegated to software?
- How can the architecture detect proxy failure before a capability causes harm?
- Can an adaptive system support competing legitimate goals without forcing them into one scalar objective?

KEY SOURCES

This chapter draws primarily on [1], [2], [4], [9], [10].

CHAPTER 03

Definitions

A precise vocabulary for observations, organizations, capabilities, and agents.

Why Definitions Matter

The terms used in intelligent software are unstable. An agent may mean a language model prompt, a background job, a human role, or a legal representative. Memory may refer to a chat transcript, a database, or a learned parameter. Without precise definitions, architecture discussions become agreements between different meanings. This chapter establishes the normative vocabulary for Volume I.

DEFINITION 3.1

Organization

A bounded but permeable network of people, resources, roles, goals, constraints, decisions, relationships, and routines coordinated toward one or more purposes.

DEFINITION 3.2

Work Object

An identifiable entity around which organizational work is coordinated, such as a customer, project, job, asset, document, location, case, order, unit, or contract.

DEFINITION 3.3

State

A representation of selected organizational facts believed to hold at a specified time. State is always partial and model-relative.

DEFINITION 3.4

Event

A recorded occurrence that asserts a change, action, decision, or observation relevant to organizational state.

DEFINITION 3.5

Observation

A timestamped measurement or assertion describing some aspect of organizational reality, together with its source, context, and provenance.

DEFINITION 3.6

Outcome

A consequential result associated with work, a decision, or a capability. Outcomes may be intended or unintended, immediate or delayed, quantitative or qualitative.

DEFINITION 3.7

Goal

An authorized description of a desired class of future states or outcomes.

DEFINITION 3.8

Intent

A context-specific commitment or direction adopted by an actor in pursuit of a goal. Intent is narrower and more immediate than purpose or goal.

DEFINITION 3.9

Constraint

A condition that limits permissible states, transitions, decisions, resources, or actions.

DEFINITION 3.10

Workflow

A constrained graph of possible state transitions through which work objects may move. A workflow describes permitted structure, not necessarily actual behavior.

DEFINITION 3.11

Decision

A commitment by an authorized actor that selects, rejects, ranks, or defers one or more possible actions or states.

DEFINITION 3.12

Knowledge

Information whose meaning, provenance, and applicability are sufficiently established for an actor to use it in reasoning or action.

DEFINITION 3.13

Understanding

A model that explains relevant relationships well enough to support prediction, interpretation, or intervention across more than one instance.

DEFINITION 3.14

Memory

A governed mechanism by which prior observations, decisions, models, and outcomes remain available for later retrieval and use.

DEFINITION 3.15

Authority

The legitimate and machine-enforceable scope within which an actor may access information, make decisions, commit resources, or change organizational state.

DEFINITION 3.16

Capability

A persistent organizational ability that reduces the resources, uncertainty, coordination, or expertise required to achieve a class of outcomes.

DEFINITION 3.17

Capability Definition

A versioned specification of the knowledge, authority, inputs, constraints, tools, procedures, tests, and outcome measures required to realize a capability.

DEFINITION 3.18

Agent

An identifiable decision-making runtime that pursues goals, uses knowledge and tools, and acts within explicitly defined authority and capability boundaries.

DEFINITION 3.19

Agent Instance

A particular execution context of an agent, including its identity, active mandate, working memory, tools, state, and audit history.

DEFINITION 3.20

Policy

A normative rule that constrains decisions or actions across a class of situations and is associated with an accountable authority.

DEFINITION 3.21

Exception

An observed situation in which actual work cannot proceed safely or effectively through the expected workflow or policy.

DEFINITION 3.22

Friction

Repeated expenditure of avoidable time, coordination, attention, rework, or uncertainty in pursuit of an outcome.

Relationships Between the Terms

An organization pursues goals through actors working on work objects. Events and observations provide evidence about changing state. Workflows describe anticipated transitions; actual event histories reveal what occurred. Constraints and policies limit actions. Decisions express intent under authority. Outcomes provide evidence about effectiveness. Memory preserves this evidence. Capabilities turn recurring knowledge and coordinated action into durable ability. Agents may realize capabilities at runtime.

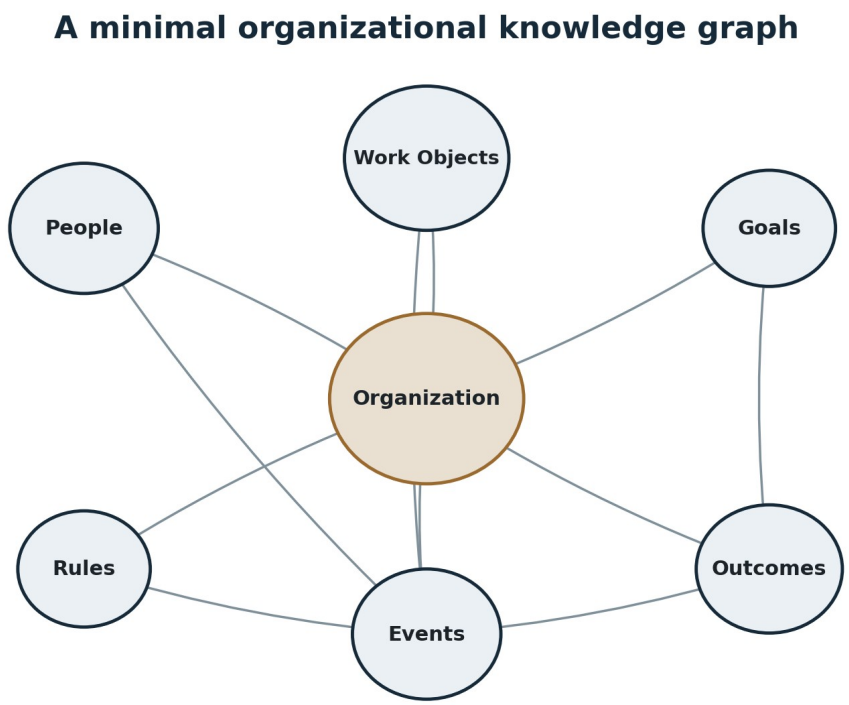


Figure 3.1 - A minimal graph of organizational entities. A production model would include temporal validity, provenance, and domain-specific relationships.

Important Distinctions

Event versus observation

An event is a recorded occurrence in the system. An observation is an assertion about reality. A user submitting a completion form is an event; the claim that the physical work is complete is an observation carried by that event. Conflating the two causes software to treat recorded input as unquestionable fact.

Workflow versus process history

A workflow is normative or expected structure. A process history is empirical. The distinction allows the system to compare intended work with actual work and identify exceptions without erasing either.

Agent versus model

A model transforms inputs into outputs. An agent is a governed runtime system that may use one or more models but also possesses identity, memory, tools, policies, and authority. A language model alone is not an agent.

Automation versus capability

Automation executes a procedure. Capability includes the surrounding ability: recognizing when the procedure applies, acquiring inputs, handling exceptions, using authority, measuring outcomes, and maintaining the procedure over time.

Glossary Discipline

These definitions are intentionally narrow. Future volumes may introduce subtypes such as human agent, software agent, institutional memory, composite capability, or latent constraint. New terms SHOULD extend rather than overload the existing vocabulary. When a term is revised, the manuscript should record the previous definition and explain the change.

Chapter Summary

The vocabulary establishes a chain from organizational reality to computational action. Observations and events represent changing state; goals, constraints, policies, and authority shape legitimate decisions; memory supports learning; capability describes durable ability; and agents realize capabilities at runtime.

Open Research Questions

- Can the distinction between observation and event be maintained consistently in ordinary application interfaces?
- What minimum provenance is required before an observation may influence an automated decision?
- How should conflicting observations coexist in organizational memory?
- What makes a capability persistent when its people, models, or tools change?

KEY SOURCES

This chapter draws primarily on [1], [18], [19], [20].

CHAPTER 04

Mathematical Models

Graphs, state transitions, time, constraints, optimization, and information.

Purpose of Formalization

A mathematical model is not the organization. It is a deliberately incomplete representation that makes selected questions tractable. Formalization is useful when it exposes hidden assumptions, permits consistency checks, supports simulation, or enables computation. It becomes dangerous when mathematical neatness is mistaken for organizational truth.

Organizations as Temporal Attributed Graphs

A useful baseline is a temporal attributed multigraph. Vertices represent actors, work objects, goals, policies, capabilities, and other entities. Edges represent typed relationships. Attributes represent values whose meaning and validity may change over time.

```
G_t = (V_t, E_t, A_t, P_t)
```

Equation 4.1 - Organizational graph at time t: vertices, edges, attributes, and provenance.

The subscript t matters. A person can change roles, a contract can expire, a policy can be superseded, and a dependency can exist only during a project phase. A graph without temporal validity collapses historical and current truth. RDF provides a general graph model for linked statements, while OWL adds formal ontology constructs; neither alone supplies the complete temporal and operational model required here. [18][19]

HYPOTHESIS H-001

Organizations are usefully represented as dynamic knowledge graphs

A temporal knowledge graph can unify entities, relationships, provenance, and changing organizational context sufficiently well to support discovery and governed automation.

State and Transition Systems

For a bounded workflow, let S be the set of representable states, A the set of actions, and T a transition relation. A transition is permitted when relevant constraints are satisfied and the actor possesses authority.

```
T : (state, action, context, authority) -> {next_state, rejected,
                                            deferred}
```

Equation 4.2 - A transition is conditional on more than the current state and action.

This model differs from a simple finite-state machine in two ways. First, context includes external facts, time, dependencies, and evidence. Second, rejection and deferral are meaningful results. A system should record why a transition did not occur rather than treating failure as an absence of data.

Events and Temporal Order

Distributed systems do not share a perfectly observed global clock. Lamport showed that event ordering requires logical relationships, not merely timestamps. Organizational event histories face the same issue: an offline mobile submission, a calendar change, and a database update may be recorded in an order different from the real-world actions they describe. [17]

Each event SHOULD therefore distinguish at least occurrence time, observation time, recording time, and processing time when these differences matter. Causal links, correlation identifiers, and supersession relations are often more reliable than sorting by one timestamp.

```
e = (id, type, subject, actor, t_occurs, t_records, source, payload,
                                  provenance)
```

Equation 4.3 - Minimal canonical event structure.

Constraints

Constraints may be hard or soft. Hard constraints define forbidden states or actions: regulatory requirements, physical impossibility, safety rules, or authority boundaries. Soft constraints express preferences and trade-offs: minimizing travel, balancing workload, or preserving continuity. The architecture must not allow an optimizer to trade away a hard constraint because it improves a score.

```
            maximize  U(outcomes) - Cost(resources) - Risk
        subject to  C_hard(x) = true  and  penalties(C_soft(x))
```

Equation 4.4 - A simplified constrained decision objective.

In practice, utility is rarely a single scalar. Organizations pursue multiple goals held by different stakeholders. The system may use multi-objective optimization, Pareto frontiers, lexicographic priorities, or explicit human choice. The formalism should reveal trade-offs rather than conceal them behind a composite score.

Uncertainty and Information

Shannon entropy measures uncertainty in a probability distribution, not organizational meaning. It is nevertheless useful for asking which observation would reduce uncertainty about a decision or process. [11]

```
                H(X) = - sum_x p(x) log p(x)
```

Equation 4.5 - Shannon entropy of a discrete variable.

An observation has high information value when it changes the distribution of plausible explanations or actions. This suggests a practical design question: which additional field, sensor, or human annotation would most improve a model? The answer may justify data collection; collecting everything merely because storage is cheap does not.

A Capability Value Model

Capability value can be approximated as the difference between outcome-adjusted resource requirements before and after the capability, discounted by risk, maintenance, and adoption cost.

```
V(C) = E[Delta Outcome + Delta Time + Delta Cost + Delta Risk] - Build -
                              Operate - Govern
```

Equation 4.6 - A qualitative value model for capability C.

The equation is not intended to force every effect into money. Its purpose is to require explicit baselines, expected effects, and lifecycle costs. A capability that saves minutes but creates opaque authority may have negative value even when the automation appears efficient.

Counterexample: Complete Formalization

A complete formal model of an organization would require representing every relevant human intention, social relationship, physical condition, and future contingency. This is neither possible nor desirable. The architecture instead requires plural models with declared scope and confidence. Formal models should be replaceable views over evidence, not totalizing descriptions.

Chapter Summary

Formal models provide structure for reasoning about organizational graphs, state transitions, event order, constraints, uncertainty, and capability value. Their purpose is to support disciplined intervention while preserving the distinction between a model and the organization it describes.

Open Research Questions

- Which temporal graph model best supports both operational queries and historical reconstruction?
- How can hard constraints be proven to remain invariant across generated capabilities?
- Can information gain guide the design of observation interfaces without encouraging surveillance?
- How should multi-objective organizational trade-offs be represented to human decision makers?

KEY SOURCES

This chapter draws primarily on [11], [17], [18], [19], [25].

CHAPTER 05

Computational Models

How software represents organizational reality without reducing it to one database schema.

Representation as a Set of Views

No single representation serves every organizational question. Transactional tables are effective for current operational state. Event logs preserve history. Graphs expose relationships. Documents preserve narrative context. Analytical models support aggregation. Vector indexes may retrieve semantically related material. The architecture therefore adopts a plural model rather than declaring one database to be the organization.

PRINCIPLE P-008

The organizational model is plural

Event, graph, document, transactional, and analytical representations SHOULD coexist as governed projections of shared evidence.

The Canonical Event Layer

The canonical event layer is the durable record of meaningful occurrences and observations. Events are immutable after acceptance; corrections are expressed through superseding or compensating events. This protects historical reasoning and enables new projections to be built later. Event sourcing is one implementation pattern, but the deeper principle is that history must not disappear when current state changes. [21][22]

A canonical event envelope should include identity, type, subject, actor, timestamps, source system, correlation and causation identifiers, schema version, tenant, sensitivity classification, payload, and provenance. CloudEvents provides a useful interoperable envelope pattern, though domain semantics remain the responsibility of the platform. [27]

```
event -> validate -> append -> project -> observe outcome -> evaluate
```

Figure 5.1 - Canonical event processing sequence.

Work-Object Registry

A work-object registry assigns durable identities to the things around which work is organized. The registry prevents each module from inventing incompatible notions of customer, project, unit, asset, or document. It does not require one universal table. It requires consistent identity, type, lifecycle, and relationship semantics.

- Global identity: a stable identifier independent of display name or external system key.

- Type and schema version: the contract governing required and optional attributes.
- Temporal lifecycle: creation, activation, suspension, completion, archival, and supersession.
- Relationships: ownership, dependency, location, membership, version, and derivation.
- Provenance: who or what asserted each important fact and under which authority.

State Projections

Operational interfaces need fast access to current state. Rather than querying the entire event history for every page load, projection services maintain purpose-specific views. A scheduling projection may organize work by date and crew; a compliance projection may organize evidence by requirement; a graph projection may connect actors, objects, and decisions. Each projection is rebuildable from authoritative events or explicitly documented source data.

This model resembles the separation between write and read concerns used in event-driven systems, but the architecture must resist unnecessary complexity. Small implementations may begin with ordinary relational tables plus an append-only audit/event table. The conceptual boundary matters more than premature infrastructure.

Organizational Memory

Memory is divided by function rather than by storage technology.

- Episodic memory: specific events, decisions, exceptions, and outcomes.
- Semantic memory: stable concepts, policies, entities, and relationships.
- Procedural memory: workflows, playbooks, capability definitions, and tool instructions.
- Working memory: context assembled for an active human or agent task.
- Institutional archive: retained records whose primary purpose is legal, historical, or evidentiary.

Retrieval must respect authority and purpose. An agent that can access employee health information for one authorized workflow should not automatically use it for scheduling or performance analysis. Memory is governed context, not a universal prompt.

Ontology and Schema Evolution

Ontology engineering provides mechanisms for defining classes and relationships, but organizations change faster than centrally designed taxonomies. The model should support a stable core vocabulary and domain-specific extensions. RDF and OWL demonstrate how graph statements and formal semantics can be separated from concrete serialization. [18][19]

Schema changes MUST preserve the ability to interpret historical data. This may require event upcasters, versioned readers, migration records, or immutable snapshots of the schema used at the time of an event. The goal is not eternal backward compatibility for every application interface; it is evidentiary continuity.

Provenance and Confidence

PRINCIPLE P-004

Representations preserve provenance and time

Important facts MUST remain connected to their source, observation time, validity interval, and confidence or dispute status.

A fact may be entered by a user, imported from a customer system, inferred by a model, or generated by a sensor. These are not epistemically equivalent. The platform should present them differently and permit contradiction. A later, better observation may supersede an earlier claim without deleting the historical record.

Counterexample: The Universal JSON Document

A universal JSON blob appears flexible because it can store any shape. It becomes fragile when identity, semantics, relationships, and versioning remain implicit. Flexibility at write time transfers cost to every reader and makes governance difficult. Extensibility should occur through explicit versioned schemas and typed relationships, not through abandoning meaning.

Chapter Summary

The computational model is a governed set of projections over evidence. A canonical event layer preserves history, the work-object registry preserves identity, graphs preserve relationships, documents preserve narrative, and state projections serve operational needs. Provenance and temporal validity are mandatory structural properties.

Open Research Questions

- Which data must be authoritative events, and which can remain derived projections?
- How should conflicting source systems establish precedence without erasing disagreement?
- What minimum ontology can serve many industries without becoming meaningless?
- How can sensitive memory be retrieved for legitimate context without enabling secondary misuse?

KEY SOURCES

This chapter draws primarily on [18], [19], [21], [22], [27], [28].

CHAPTER 06

Organizational Models

How businesses actually coordinate knowledge, authority, routines, and exceptions.

Organizations as Distributed Decision Systems

An organization is not merely a hierarchy or a collection of workflows. It is a distributed decision system. People hold different knowledge, face different constraints, possess different authority, and act at different times. Coordination mechanisms - meetings, schedules, forms, markets, policies, supervision, and software - exist because no participant has complete information.

Simon's bounded rationality explains why decision makers use simplified models and satisfactory choices rather than exhaustive optimization. Hayek's analysis of dispersed knowledge explains why relevant information cannot simply be centralized in advance. Coase explains why firms form when administrative coordination can be less costly than repeated market transactions. Together, these ideas suggest that organizational architecture is largely an architecture of decision rights, information flow, and coordination cost. [5][6][12][13]

Formal and Informal Organization

The formal organization contains job descriptions, policies, reporting lines, and approved workflows. The informal organization contains trust, reputation, local expertise, favors, shortcuts, and tacit coordination. Effective software must represent the formal structure without assuming that it fully explains actual work.

A discrepancy is not automatically misconduct. It may reveal a missing exception path, an unrealistic policy, an external constraint, or valuable local innovation. The observation layer should preserve the discrepancy and its context so that humans can determine which interpretation applies.

Roles and Authority

A role is not merely a permission bundle. It combines responsibility, expected judgment, access, and accountability. Authority may derive from organizational position, professional qualification, ownership, contract, delegation, or temporary incident command. It can be conditional and time-bounded.

PRINCIPLE P-005

Authority is explicit and machine-enforceable

Every consequential action by a human or software agent MUST be evaluated against a declared source and scope of authority.

Role-based access control provides a mature starting point for assigning permissions through roles, but adaptive systems also require attributes, context, purpose, delegation, and separation of duties. [26] A scheduling agent may be authorized to propose assignments, for example, but not to override safety certifications or approve overtime above a threshold.

Routines as Organizational Memory

Routines carry accumulated knowledge about how tasks are performed and coordinated. Evolutionary theories of the firm treat routines as relatively persistent patterns that can be selected, replicated, and modified. [15] The Capability Architecture extends this insight by asking when a routine can be represented as a capability definition and how that definition changes when moved between organizations.

Not every routine is valuable. Some persist because no one owns the problem, because a legacy system requires them, or because an old constraint disappeared without the routine being reconsidered. Observation must therefore link routines to outcomes and constraints rather than equating recurrence with correctness.

Exceptions as Discovery Material

HYPOTHESIS H-005

Persistent friction appears as repeated exception handling

Before a missing capability is named, it often appears as repeated manual coordination, copying, checking, waiting, escalation, or workaround behavior.

Exception records should capture the expected path, the blocking condition, the actor's response, the cost of the response, and the outcome. A pattern of exceptions may indicate a missing workflow branch, poor data quality, a policy conflict, or a new capability opportunity.

Knowledge, Power, and Legibility

Making work legible changes the organization. A new metric can shift attention; a new workflow can move authority; a dashboard can privilege one interpretation. The system should record who requested a representation, which decisions it supports, and whose work it may affect. Ostrom's research on institutions emphasizes that durable governance depends on locally appropriate rules, monitoring, graduated responses, and mechanisms for resolving conflict rather than one universal arrangement. [14]

A Minimal Organizational Model

- Purpose and authorized goals.
- Actors, roles, qualifications, and delegated authority.
- Work objects and their relationships.
- Expected workflows, policies, and hard constraints.
- Actual events, decisions, exceptions, and outcomes.
- Resources and capacities.
- Communication and dependency structures.

- Capabilities and their owners.
- Governance processes for changing all of the above.

Counterexample: The Org Chart as the Organization

An org chart represents reporting relationships. It does not represent who possesses critical knowledge, which teams depend on one another, where work waits, how decisions are escalated, or which informal relationships make the organization function. Using the org chart as the primary computational model would encode authority while omitting operation.

Chapter Summary

Organizations are distributed decision systems whose practical intelligence is spread across formal structure, informal coordination, routines, and local knowledge. The architecture must represent authority and expected workflows while learning from actual events and exceptions.

Open Research Questions

- How can tacit knowledge be observed without forcing workers to document every judgment?
- When should an informal routine be formalized, and when should it remain flexible?
- How should the platform represent disputed authority or overlapping mandates?
- Can capability transfer preserve local governance rather than imposing the source organization's assumptions?

KEY SOURCES

This chapter draws primarily on [5], [6], [7], [12], [13], [14], [15], [26].

CHAPTER 07

Capability Models

How persistent organizational abilities emerge, evolve, compose, and become reusable intellectual property.

Capability as the Unit of Change

Features describe software behavior. Workflows describe permitted transitions. Capabilities describe what an organization can reliably accomplish. This difference makes capability a stronger unit for adaptive architecture. A feature can exist without being adopted, a workflow can exist without producing good outcomes, and an automation can execute without understanding when it should stop. Capability includes the surrounding organizational system.

HYPOTHESIS H-002

Capability is the fundamental reusable unit

A capability definition captures more transferable organizational intelligence than an isolated feature, prompt, workflow, or model.

The Anatomy of a Capability Definition

- Identity and purpose: the class of outcomes the capability exists to support.
- Inputs and required context: the observations, work objects, and evidence needed to operate.
- Knowledge: domain concepts, examples, policies, and models used in interpretation.
- Authority: permitted decisions, actions, resource commitments, and escalation boundaries.
- Constraints: safety, legal, organizational, temporal, and technical invariants.
- Procedure or strategy: deterministic steps, heuristics, planning methods, and tool use.
- Tools and dependencies: APIs, services, human roles, data sources, and subordinate capabilities.
- Outcome measures: effectiveness, cost, time, quality, risk, adoption, and unintended effects.
- Tests and simulations: conditions under which the capability is safe to deploy.
- Ownership and lifecycle: accountable maintainer, version, review period, and retirement criteria.

Capability Emergence

Capabilities can be designed deliberately, discovered from successful practice, imported from another organization, or assembled from existing components. The observation-driven path begins with recurring friction or a repeated high-performing response. The system groups related events, identifies context and outcomes, proposes a generalized model, and tests whether the model transfers beyond the original cases.

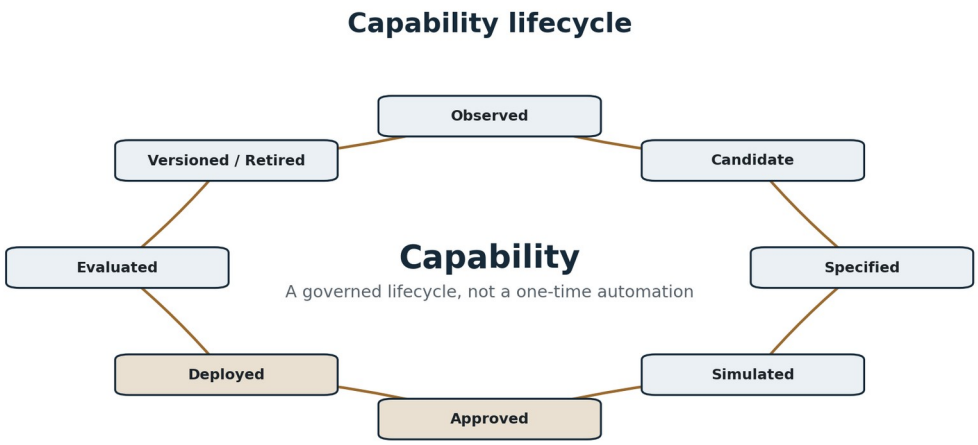


Figure 7.1 - A capability moves through an explicit lifecycle. Deployment is one stage, not the end of the process.

PRINCIPLE P-007

Capabilities require outcomes and a retirement path

Every deployed capability MUST have an accountable owner, measurable outcome model, review cadence, and conditions under which it will be revised, suspended, or retired.

Composition

A composite capability depends on subordinate capabilities. A project staffing capability may use qualification verification, availability forecasting, travel estimation, and budget authorization. Composition requires explicit contracts: what each capability guarantees, what evidence it requires, and which failures it may return.

The architecture should avoid inheritance based only on superficial similarity. A healthcare scheduling capability and a construction scheduling capability may share temporal optimization components while differing radically in safety, consent, and authority. Reuse should occur through small explicit sub-capabilities and adaptable policy layers.

Capability Versioning

Capability versions are semantically meaningful. A new version may change inputs, authority, outcome targets, policies, models, or tools. The system must preserve which version influenced each decision and permit controlled rollout, comparison, and rollback.

```
CapabilityVersion = Definition + PolicySet + Dependencies + Tests + OutcomeModel
```

Equation 7.1 - A version identifies the complete governed contract, not merely source code.

Capability as Intellectual Property

The economically valuable part of a capability is accumulated judgment: which context matters, which exceptions are dangerous, which sequence works, what evidence is sufficient, and where authority must stop. The underlying model provider may be interchangeable. This suggests that capability definitions, evaluation suites, curated organizational memory, and adaptation methods may form a distinct category of intellectual property.

HYPOTHESIS H-006

Agent value resides primarily in encoded organizational judgment

Model access is increasingly commoditized; differentiated value may reside in the capability definition, memory, policies, evaluations, and domain adaptation surrounding the model.

Licensing and Transfer

A capability marketplace is plausible only if transfer does not conceal assumptions. A licensed capability should declare its source context, supported domains, required inputs, authority model, known failure conditions, validation evidence, and adaptation obligations. Transfer resembles transplanting an institutional routine, not installing a generic plugin.

HYPOTHESIS H-007

Capabilities can become licensable components

Versioned capability definitions may support controlled reuse across organizations when dependencies, governance, and adaptation requirements are explicit.

Measuring Capability

Capability measurement should compare an organization's ability before and after deployment across multiple dimensions: outcome quality, time, cost, coordination load, error rate, resilience, explainability, and dependence on scarce expertise. Adoption is part of capability. A technically correct tool that people cannot trust or integrate into work has not created the intended ability.

Counterexample: The Prompt Library

A prompt library may be useful, but it does not by itself contain authority, memory, outcome measures, tool contracts, governance, or lifecycle. Calling prompts capabilities obscures the organizational system required for reliable action. Prompts may be implementation artifacts inside a capability definition.

Chapter Summary

Capability is the architecture's central unit: a durable, governed organizational ability rather than a software feature. Capability definitions can be discovered, composed, tested, versioned, transferred, evaluated, and retired. Their reusable value lies in encoded judgment and structure.

Open Research Questions

- What evidence is sufficient to generalize a successful local routine into a capability?
- How should capability dependencies express probabilistic or human guarantees?
- Which elements of a capability can be protected or licensed as intellectual property?
- How can transferred capabilities adapt without losing the evidence supporting their original claims?

KEY SOURCES

This chapter draws primarily on [1], [15], [16], [20], [23], [24].

CHAPTER 08

Agent Models

What agents are philosophically, organizationally, technically, and economically.

An Agent Is Not a Chat Interface

The term agent should describe an autonomous executor of intention operating within boundaries, not a particular user interface or model. A human employee can act as an agent of an organization. A background software process can act as an agent when it interprets a mandate and changes state. A language model becomes part of an agent only when it is placed inside a runtime with identity, memory, tools, policies, authority, and evaluation.

PRINCIPLE P-006

Agents receive bounded autonomy

An agent MUST operate under an explicit mandate, authority scope, tool boundary, and evaluation regime. General access is not a substitute for architecture.

Four Perspectives

Philosophical

Philosophically, an agent is an entity to which intention and action can be attributed for a defined purpose. The architecture does not require metaphysical claims about consciousness. It requires operational accountability: which entity selected an action, under which mandate, using which evidence?

Practical

Practically, agents correspond to organizational roles. A scheduling agent, procurement agent, compliance agent, or project-intake agent has responsibilities and limits analogous to a role description. This framing makes agent design legible to non-specialists and connects software autonomy to established governance.

Technical

Technically, an agent is a runtime composition. Modern research systems combine language models with memory, planning, reflection, and external tools. ReAct interleaves reasoning and action; generative-agent architectures combine observations, memory, reflection, and planning. These systems demonstrate useful patterns but do not by themselves provide organizational authority or governance. [23][24]

Economic

Economically, the agent is a deployment of organizational knowledge. Its differentiated value may lie in the capability it realizes, not in the general-purpose model. This separates model infrastructure from organizational intellectual property.

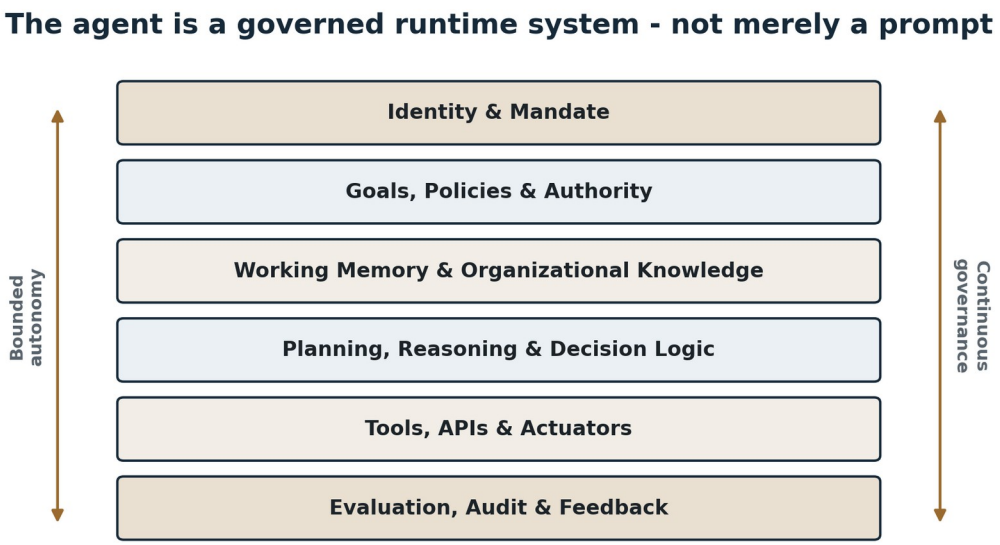


Figure 8.1 - A governed agent is a layered runtime. Models may participate in reasoning, but authority and evaluation surround them.

Agent Runtime Components

- Identity: a stable actor identifier used in events, decisions, and audits.
- Mandate: the active task, goal, or responsibility the agent is authorized to pursue.
- Authority: resources and state changes the agent may commit.
- Knowledge: domain material and organizational context available for reasoning.
- Memory: retrieved historical evidence and current working context.
- Policies: rules the agent must apply regardless of model preference.
- Planner: mechanism for decomposing work and selecting actions.
- Tools: typed interfaces for reading or changing external systems.
- Evaluator: deterministic checks, model-based critique, simulation, or human review.
- Audit stream: a record of inputs, decisions, actions, versions, and outcomes.

Capability Definition and Runtime Agent

HYPOTHESIS H-003

Agents are runtime realizations of capability definitions

The capability definition is the reusable governed specification; an agent instance is one execution of that specification in a particular context.

This distinction is analogous to class and object, process definition and process instance, or executable and running process, though none is exact. It allows one capability to be realized by different agent architectures and one agent runtime to load different capability definitions.

Planning and Tool Use

Agent planning should be constrained by typed tools and explicit state transitions. Free-form generation may propose a plan, but consequential actions should pass deterministic validation. Tool contracts should define inputs, outputs, errors, idempotency, side effects, permissions, and compensation behavior.

Where possible, planning should be separated from commitment. An agent MAY prepare a proposed schedule, contract, or message using broad context while a narrower authorized component commits the change after validation or approval.

Memory and Context

More context is not always better. Excess context increases cost, exposes unrelated sensitive material, and may confuse the model. Agent memory should be purpose-limited and assembled through governed retrieval. Working memory should identify why each item is present and which source supports it.

Multi-Agent Systems

Multiple agents may specialize, review, or negotiate. Specialization can improve modularity, but it also creates coordination cost, ambiguity, and opportunities for responsibility to diffuse. A multi-agent design should be justified by distinct authority, knowledge, or lifecycle boundaries rather than by the appeal of simulated teamwork.

Failure Modes

- Authority leakage: the agent gains indirect access to actions outside its mandate.
- Context contamination: irrelevant or sensitive memory affects a decision.
- Goal drift: optimization follows a proxy rather than the authorized purpose.
- Compounding error: one uncertain inference becomes accepted context for later actions.
- Unbounded retries: the agent consumes resources or repeats harmful actions.
- Invisible delegation: a user cannot tell which component made a consequential choice.
- Evaluation capture: the same model or logic both produces and approves its action.

Counterexample: The Universal Executive Agent

A universal agent with access to every system, every memory, and broad instructions to improve the business is architecturally indefensible. It has no meaningful authority boundary, no stable evaluation target, and no accountable capability contract. Apparent generality is produced by removing the safeguards that make action legitimate.

Chapter Summary

Agents are governed runtime systems that realize capabilities. Their structure includes identity, mandate, authority, memory, knowledge, planning, tools, and evaluation. The model is one replaceable component. Bounded autonomy, typed action, and auditable responsibility are foundational.

Open Research Questions

- How should agent identity persist across model, tool, and capability upgrades?
- Which decisions require independent evaluation rather than self-critique?
- How can human supervisors understand an agent's active authority at a glance?
- When does multi-agent specialization improve reliability enough to justify coordination complexity?

KEY SOURCES

This chapter draws primarily on [23], [24], [26], [29].

CHAPTER 09

Architecture

How observation, organizational models, capability definitions, agents, and governance fit together.

Architectural Objective

The architecture must support two modes simultaneously. It must operate ordinary business software reliably today, and it must preserve enough evidence and structure to improve that software tomorrow. A platform that only experiments cannot become trusted infrastructure. A platform that only executes fixed workflows cannot become adaptive.

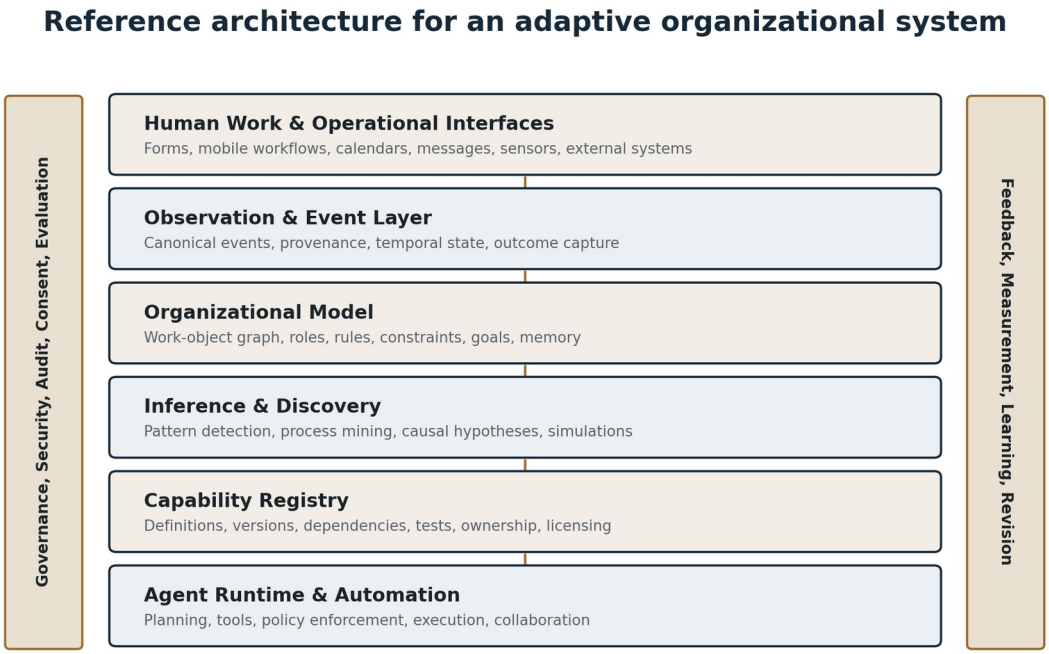


Figure 9.1 - The reference architecture separates operational work, evidence, organizational modeling, discovery, capability definitions, and runtime execution. Governance and evaluation cross every layer.

Layer 1: Human Work and Operational Interfaces

The top layer is where people and external systems perform work: mobile forms, calendars, project views, messages, sensors, imports, and APIs. Interfaces should minimize burden while capturing meaningful context. They must support ordinary exceptions rather than forcing users to falsify data to continue.

Layer 2: Observation and Event

The observation layer converts operational interactions into canonical events and observations. It validates schema and authority, assigns provenance, links related actions, and records temporal information. It should accept evidence from humans, systems, and models while preserving the distinction between them.

Layer 3: Organizational Model

The organizational model maintains work-object identity, relationships, roles, constraints, goals, policies, memory, and current projections. It is not one database. It is a set of governed models connected through stable identity and evidence.

Layer 4: Inference and Discovery

Discovery services search for recurring patterns, process variants, bottlenecks, outcome correlations, and contradictory rules. Some analyses are deterministic; others use statistical or language models. Results are hypotheses, not automatic truth. The layer should produce evidence packages that a human can inspect.

Layer 5: Capability Registry

The registry stores versioned capability definitions, dependencies, owners, authority requirements, tests, evaluations, known limitations, and deployment status. It is the bridge between discovered insight and operational change. A proposed capability cannot enter production merely because code or a prompt was generated.

Layer 6: Agent Runtime and Automation

The runtime executes approved capabilities through deterministic services, workflow engines, human tasks, and model-assisted agents. It enforces tool contracts and authority, records actions as events, and returns outcomes to the observation layer.

Cross-Cutting Governance

Governance includes identity, access, consent, data classification, policy enforcement, change approval, audit, incident response, and model evaluation. Governance is not a layer below intelligence; it is the structure that makes intelligence usable inside an organization.

Control Plane and Operational Plane

The operational plane processes day-to-day work. The control plane defines schemas, policies, capabilities, tool permissions, versions, evaluation thresholds, and rollout. Separating them allows capabilities to be tested and approved without modifying production data directly.

Multi-Tenancy and Isolation

A platform serving multiple organizations must treat each tenant's data, capability adaptations, and learned patterns as separate by default. Shared capability definitions should not imply shared organizational memory. Cross-tenant learning requires explicit mechanisms such as de-identified aggregate evidence, licensed artifacts, or federated evaluation.

Reliability and Degradation

Core operations should degrade safely when model services are unavailable. Deterministic workflows, identity, authorization, event capture, and existing capabilities should remain functional. Model-assisted discovery may pause; the business should not.

- Every write path is idempotent or supports compensation.
- Every consequential model decision is linked to model and prompt/configuration versions.
- Every projection can be rebuilt or reconciled against authoritative evidence.
- Every capability supports staged rollout and rollback.
- Every agent action has a bounded timeout, retry policy, and escalation path.
- Every sensitive retrieval is purpose-scoped and auditable.

Evolutionary Deployment

The architecture should be adopted in stages. First, improve identity and event capture. Second, visualize actual processes. Third, formalize rules and deterministic automations. Fourth, introduce model-assisted discovery and summarization. Fifth, deploy bounded agents for narrow capabilities. Sixth, permit capability generation only where evaluation and governance are mature.

Counterexample: AI Layer on Top

Adding a language-model chat box to a conventional application does not create an adaptive architecture. The model may summarize or answer questions, but it lacks a faithful organizational model, capability lifecycle, authority semantics, and outcome feedback. Intelligence cannot be safely layered on top of missing structure.

Chapter Summary

The reference architecture separates evidence, organizational representation, discovery, capability governance, and runtime execution while allowing feedback between them. Governance, security, and evaluation cross every layer. The system is designed to begin as reliable software and grow into adaptive capability development.

Open Research Questions

- Which layers can be combined in the first implementation without destroying their conceptual boundaries?
- How should cross-tenant capability learning be governed and compensated?
- What evidence threshold should permit a discovered pattern to become a capability candidate?
- How can the platform prove that a capability rollout can be safely reversed?

KEY SOURCES

This chapter draws primarily on [1], [17], [18], [19], [22], [23], [24], [27], [28].

CHAPTER 10

Implementation

A practical technical foundation using current software while preserving architectural independence.

Implementation Philosophy

The reference implementation is intentionally conservative. The first objective is not to demonstrate maximum autonomy. It is to establish trustworthy identity, events, organizational state, authority, and evaluation. Advanced agents are only useful when the underlying system can explain what changed and why.

Reference Technology Stack

A credible first implementation can be built with a small set of well-understood technologies:

- Go services for APIs, event ingestion, policy enforcement, background work, and deterministic capability components.
- React for operational interfaces, process visualization, review workflows, and human approvals.
- PostgreSQL for transactional state, event metadata, identities, roles, and capability registry data.
- Object storage for documents, images, model artifacts, and immutable evidence.
- A graph projection - initially relational adjacency tables or a dedicated graph database later - for relationship-heavy queries.
- A queue or event broker when asynchronous scale justifies it; a database-backed outbox is sufficient initially.
- Containerized services with persistent managed storage. Containers are replaceable compute, not data persistence.
- A model gateway that centralizes provider access, redaction, prompt/configuration versions, rate limits, and evaluation.
- OpenTelemetry-compatible traces, metrics, and logs for operational observability. [28]

Bounded Contexts

The codebase should be divided around stable domain responsibilities rather than technical layers alone.

- Identity and Authority: users, services, roles, delegations, qualifications, policy decisions.
- Organization Registry: organizations, work objects, relationships, and lifecycle.
- Observation: event ingestion, schema registry, provenance, attachments, and corrections.
- Workflow: state models, tasks, constraints, exceptions, and human approvals.
- Memory: documents, semantic retrieval, retention, sensitivity, and evidence links.
- Capability Registry: definitions, versions, dependencies, tests, owners, deployment state.

- Agent Runtime: mandates, sessions, context assembly, tool invocation, and audits.
- Evaluation: baselines, outcome capture, experiments, rollouts, incidents, and retirement.

Core Data Structures

```
WorkObject { id, tenant_id, type, schema_version, lifecycle, attributes }
```

Listing 10.1 - Conceptual work-object record.

```
Event { id, tenant_id, type, subject_id, actor_id, occurred_at,
  recorded_at, source, schema_version, payload, provenance }
```

Listing 10.2 - Conceptual canonical event.

```
Capability { id, version, purpose, inputs, authority, constraints,
  strategy, tools, dependencies, tests, outcomes, owner, status }
```

Listing 10.3 - Conceptual capability definition.

```
AgentRun { id, capability_version, mandate, actor_id, context_refs,
  tool_calls, decisions, result, evaluation }
```

Listing 10.4 - Conceptual agent execution record.

API Boundaries

APIs should expose domain actions rather than unrestricted database mutation. For example, POST /work-objects/{id}/complete should validate authority, required evidence, workflow state, and idempotency before emitting a completion event. The endpoint should not simply set status = complete.

Tools exposed to agents should be narrower still. A tool named propose_schedule may return a candidate without committing calendar events. A separate commit_schedule tool may require a signed approval token and deterministic constraint validation.

Event Processing and Projections

A transactional outbox can atomically store domain state changes and pending events in PostgreSQL. A worker publishes or processes the event, updating projections and analytics. This avoids requiring distributed transactions in the first implementation. As scale grows, an event broker can replace or extend the mechanism without changing canonical event semantics. [22][27]

Policy Enforcement

Authority checks should be centralized enough to remain consistent but expressed through domain-specific decisions. A policy engine receives actor, action, resource, context, and purpose, then returns permit, deny, or require-approval with reasons. Decisions are recorded as events.

IMPLEMENTATION PATTERN IP-001

Plan broadly, commit narrowly

Model-assisted components MAY use rich context to prepare proposals. Consequential state changes SHOULD pass through narrow deterministic commands that enforce policy and constraints.

Model Gateway

All model calls should pass through a gateway that records provider, model version, configuration, capability version, purpose, input references, output, latency, cost, and evaluation metadata. Sensitive content should be minimized or transformed before transmission according to policy. The gateway enables provider substitution and prevents model integration from leaking throughout the domain code.

Testing and Evaluation

- Unit tests for deterministic constraints, permissions, transitions, and calculations.
- Contract tests for tools and external integrations.
- Replay tests that rebuild projections from historical events.
- Simulation tests for capability definitions using representative and adversarial cases.
- Golden-set evaluations for model-assisted classification, extraction, planning, and review.
- Shadow deployment in which a new capability produces recommendations without acting.
- Canary rollout to bounded users, projects, or tenants.
- Outcome comparison against an explicit baseline and monitoring for unintended effects.

Observability

Operational observability and organizational observation are related but distinct. Traces, metrics, and logs explain software behavior. Organizational events explain work behavior. They should share correlation identifiers where useful, but debugging telemetry must not silently become employee performance data. OpenTelemetry provides common conventions for software signals; the organizational event model adds domain meaning and governance. [28]

Development Roadmap

Stage 0 - Stable operational core

Build identity, tenant isolation, work-object registry, current-state workflows, and persistent storage. Avoid agents.

Stage 1 - Observation layer

Introduce canonical events, provenance, corrections, outcome fields, and structured exception capture. Make event history visible to users.

Stage 2 - Process understanding

Build process timelines, variant comparison, bottleneck analysis, dependency graphs, and searchable organizational memory.

Stage 3 - Deterministic capabilities

Convert stable rules and recurring procedures into versioned capability definitions backed by deterministic services and human approvals.

Stage 4 - Assisted discovery

Use models to cluster exceptions, summarize evidence, propose candidate rules, and draft capability definitions. Humans approve all operational changes.

Stage 5 - Bounded agents

Deploy agents for narrow, measurable capabilities with typed tools, limited authority, shadow evaluation, and reliable fallback.

Stage 6 - Capability generation

Allow the system to propose and implement new software modules only where the registry, simulations, governance, and outcome evaluation are sufficiently mature.

A Concrete First Product

The first product should appear ordinary: projects, customers, jobs, tasks, schedules, notes, documents, statuses, and reports. Its differentiator is structural. Every interaction produces governed observations. Work objects share identity. Exceptions and outcomes are first-class. Rules are versioned. Over time, the system can explain how work actually occurs and identify where a new capability would matter.

Counterexample: Starting With Autonomous Code Generation

Beginning with an agent that watches users and writes production code would maximize novelty while minimizing knowledge and control. The system would lack reliable events, a domain model, authority, baselines, and evaluation. Code generation belongs near the end of the roadmap because software creation is not the difficult part; deciding what should exist, under which constraints, and whether it improved the organization is the difficult part.

Chapter Summary

The implementation begins with conservative, auditable software: stable identity, canonical events, plural organizational models, explicit authority, and versioned capabilities. Go, React, PostgreSQL, containers, model gateways, and observability provide a practical foundation, but the architecture remains independent of them.

Open Research Questions

- What is the smallest schema that captures useful exceptions and outcomes without burdening users?
- When does a relational graph projection cease to be sufficient?
- Which model-assisted discovery tasks provide value before agents receive action authority?
- How can capability evaluations be reused across customers without leaking organizational data?

KEY SOURCES

This chapter draws primarily on [22], [26], [27], [28], [29].

CONCLUSION

From Software Products to Capability Systems

The architecture developed in this volume begins with a change in the object of software design. The conventional object is an application that encodes a workflow. The proposed object is an organizational capability system that observes work, represents changing context, develops governed interventions, and measures whether the organization became more able to achieve its purposes.

This shift does not eliminate applications, databases, workflows, or human management. It reorganizes them around a longer learning loop. Operational software becomes the sensory and actuating surface of the organization. Event histories become evidence. Organizational models become provisional explanations. Capability definitions become versioned units of change. Agents become bounded runtime realizations. Governance determines which forms of autonomy are legitimate.

The strongest claims remain hypotheses. It is not yet established that organizations can be represented adequately as temporal knowledge graphs, that capability definitions will transfer between firms, or that observation can reliably produce useful software requirements. Those questions require prototypes, case studies, failures, and formal work. The value of the architecture at this stage is that it makes the questions precise enough to investigate.

The first implementation should therefore be judged less by how intelligent it appears and more by whether it creates trustworthy organizational memory, exposes real process structure, reduces repeated friction, and supports safer change. If those foundations hold, capability generation becomes a credible direction rather than an AI demonstration.

RESEARCH PROGRAM

The next work

Develop formal capability semantics; build the observation-layer prototype; define a capability interchange format; test event and exception capture in real businesses; construct evaluation methods; and document where the theory fails.

APPENDIX A

Glossary

Term	Definition
Agent	An identifiable decision-making runtime operating under a mandate and explicit authority.
Authority	Legitimate, machine-enforceable scope for access, decision, commitment, or state change.
Capability	A persistent ability to achieve a class of outcomes with reduced resources, uncertainty, coordination, or expertise.
Capability Definition	The versioned governed specification required to realize a capability.
Constraint	A condition limiting permissible states, actions, resources, or decisions.
Decision	An authorized commitment that selects, rejects, ranks, or defers possible actions or states.
Event	A recorded occurrence relevant to organizational state.
Exception	A condition in which expected workflow or policy cannot proceed safely or effectively.
Friction	Repeated avoidable expenditure of time, attention, coordination, rework, or uncertainty.
Goal	An authorized description of desired future states or outcomes.
Intent	A context-specific commitment adopted in pursuit of a goal.
Knowledge	Information with sufficient meaning, provenance, and applicability for reasoning or action.
Memory	A governed mechanism for preserving and retrieving prior evidence, models, decisions, and outcomes.
Observation	A timestamped assertion about organizational reality with source, context, and provenance.
Organization	A network of people, resources, roles, goals, constraints, decisions, relationships, and routines coordinated toward purpose.
Outcome	A consequential result associated with work, a decision, or a capability.
Policy	A normative rule constraining decisions or actions under an accountable authority.
State	A partial, model-relative representation of facts believed to hold at a time.

Understanding	A model sufficient to explain, predict, interpret, or intervene across multiple instances.
Workflow	A constrained graph of possible state transitions.
Work Object	An identifiable entity around which work is coordinated.

APPENDIX B

Hypothesis Register

H-001

Organizations can be represented usefully as dynamic, temporal knowledge graphs.

Status: Under investigation. Evidence, contradictions, experiments, and revision history will be expanded in later drafts.

H-002

Capability is a stronger unit of reusable organizational intelligence than application feature or workflow.

Status: Tentatively supported. Evidence, contradictions, experiments, and revision history will be expanded in later drafts.

H-003

Agents are runtime realizations of capability definitions.

Status: Speculative. Evidence, contradictions, experiments, and revision history will be expanded in later drafts.

H-004

Structured observation can reveal software requirements that users cannot articulate directly.

Status: Under investigation. Evidence, contradictions, experiments, and revision history will be expanded in later drafts.

H-005

Most persistent operational friction appears first as repeated exception handling.

Status: Under investigation. Evidence, contradictions, experiments, and revision history will be expanded in later drafts.

H-006

The economic value of an agent is primarily encoded organizational judgment, not model access.

Status: Speculative. Evidence, contradictions, experiments, and revision history will be expanded in later drafts.

H-007

A capability registry can support composition, licensing, and cross-industry adaptation.

Status: Speculative. Evidence, contradictions, experiments, and revision history will be expanded in later drafts.

H-008

Outcome-linked event histories can reduce the cost of organizational learning.

Status: Under investigation. Evidence, contradictions, experiments, and revision history will be expanded in later drafts.

WORKS CITED

Sources Used in Volume I

[1] S. Burman, The Capability Architecture: Discovery Notes, unpublished working manuscript, 2026.

[2] N. Wiener, Cybernetics: Or Control and Communication in the Animal and the Machine. Cambridge, MA: MIT Press, 1948.

[3] L. von Bertalanffy, General System Theory: Foundations, Development, Applications. New York: George Braziller, 1968.

[4] S. Beer, Brain of the Firm, 2nd ed. Chichester, UK: John Wiley & Sons, 1981.

[5] H. A. Simon, Administrative Behavior, 4th ed. New York: Free Press, 1997. Original work published 1947.

[6] H. A. Simon, “A Behavioral Model of Rational Choice,” Quarterly Journal of Economics, vol. 69, no. 1, pp. 99-118, 1955.

[7] J. G. March and H. A. Simon, Organizations. New York: John Wiley & Sons, 1958.

[8] P. M. Senge, The Fifth Discipline: The Art and Practice of the Learning Organization. New York: Doubleday, 1990.

[9] D. C. Engelbart, Augmenting Human Intellect: A Conceptual Framework, SRI Summary Report AFOSR-3233, Stanford Research Institute, 1962.

[10] J. C. R. Licklider, “Man-Computer Symbiosis,” IRE Transactions on Human Factors in Electronics, vol. HFE-1, pp. 4-11, 1960.

[11] C. E. Shannon, “A Mathematical Theory of Communication,” Bell System Technical Journal, vol. 27, pp. 379-423 and 623-656, 1948.

[12] R. H. Coase, “The Nature of the Firm,” Economica, vol. 4, no. 16, pp. 386-405, 1937.

[13] F. A. Hayek, “The Use of Knowledge in Society,” American Economic Review, vol. 35, no. 4, pp. 519-530, 1945.

[14] E. Ostrom, Governing the Commons: The Evolution of Institutions for Collective Action. Cambridge, UK: Cambridge University Press, 1990.

[15] R. R. Nelson and S. G. Winter, An Evolutionary Theory of Economic Change. Cambridge, MA: Belknap Press of Harvard University Press, 1982.

[16] D. J. Teece, G. Pisano, and A. Shuen, “Dynamic Capabilities and Strategic Management,” Strategic Management Journal, vol. 18, no. 7, pp. 509-533, 1997.

[17] L. Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System,” Communications of the ACM, vol. 21, no. 7, pp. 558-565, 1978.

[18] R. Cyganiak, D. Wood, and M. Lanthaler, eds., RDF 1.1 Concepts and Abstract Syntax, W3C Recommendation, Feb. 25, 2014.

[19] W3C OWL Working Group, OWL 2 Web Ontology Language Document Overview, 2nd ed., W3C Recommendation, Dec. 11, 2012.

[20] E. Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston, MA: Addison-Wesley, 2003.

[21] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA: Addison-Wesley, 2002.

[22] M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA: O’Reilly Media, 2017.

[23] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” in Proc. International Conference on Learning Representations, 2023.

[24] J. S. Park et al., “Generative Agents: Interactive Simulacra of Human Behavior,” in Proc. 36th Annual ACM Symposium on User Interface Software and Technology, 2023, article 2, pp. 1-22.

[25] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA: MIT Press, 2018.

[26] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, “Proposed NIST Standard for Role-Based Access Control,” ACM Transactions on Information and System Security, vol. 4, no. 3, pp. 224-274, 2001.

[27] Cloud Native Computing Foundation, CloudEvents Specification, version 1.0.2, 2022.

[28] OpenTelemetry Authors, OpenTelemetry Specification, Cloud Native Computing Foundation, continuously maintained specification.

[29] S. Bradner, “Key words for use in RFCs to Indicate Requirement Levels,” RFC 2119, Internet Engineering Task Force, Mar. 1997.

REVISION NOTES

Working Draft v0.3

- Expanded the original discovery outline into ten substantive chapters.
- Added normative definitions, principles, hypotheses, implementation patterns, and research questions.
- Introduced mathematical, computational, organizational, capability, agent, and layered architecture models.
- Added a staged reference implementation using Go, React, PostgreSQL, containers, model gateways, and governed agent tools.
- Added diagrams, glossary, hypothesis register, and a numbered works-cited section.
- This draft prioritizes conceptual coherence and reader review. Formal proofs, case studies, empirical evaluations, and a complete capability interchange specification remain future work.

End of Volume I - Foundations